

FUNDAMENTAL OF WEB TECHNOLOGIES

Course Code: BCSE-012 | Maharishi Markandeshwar (Deemed to be University), Mullana (Ambala)
Comprehensive Master Notes: Complete Theory, 12-Point Comparative Matrices, Vector Architectural Schematics & University Solved Examination Questions

UNIT I: Internet & World Wide Web (6h)

Technical definitions, 12-pt comparative matrix (Internet vs WWW), Evolution (ARPANET 1969 to Web 3.0), Application spectrum, 5-tier DNS resolution flowchart, Browser subsystems & Web servers, HTTP transaction cycle, and Inverted Index search engine crawlers.

UNIT II: Basics of HTML & Engineering (6h)

8-stage SDLC lifecycle flowchart, Hosting architectures, Static vs Dynamic 12-pt comparison, HTML5 document tree DOM hierarchy, Core text formatting, Ordered/Unordered lists, Anchor hyperlinks, and Client-side image map coordinate geometry.

UNIT III: HTML5 & Modern Web APIs (6h)

HTML5 paradigm shift & HTML4 limitations, Structural semantic page layout blueprint (<header>, <nav>, <main>, <article>, <section>, <aside>, <footer>), Native multimedia multi-codec fallback, Canvas 2D graphics API, and Web Storage 3-tier decision matrix.

UNIT IV: CSS Architecture & Layouts (6h)

Cascade & specificity formula matrix, 3 CSS applying methods, Typography & color gradients, Standard CSS box model architecture & box-sizing resets, 5 CSS positioning schemes, Micro-Clearfix layout hacks, and Professional zebra-striped data tables.

💡 **University Examination Oriented:** Enriched with native vector flowcharts and solved university examination questions strictly aligned with the MMDU examination pattern (Section A: 12×1M, Section B: 4×2M, Section C: 4×4M, Section D: 4×6M) featuring diagrams in **both the question and the step-by-step solution.**

UNIT I: INTRODUCTION TO INTERNET AND WORLD WIDE WEB

Architecture, Historical Evolution, Internet vs WWW, Protocol Stacks, Web Browsers, Web Servers, HTTP & Web-Casting (6 Hours)

1. Technical Definitions: Internet vs. World Wide Web

TECHNICAL DEFINITION: THE INTERNET

A globally coordinated, decentralized network of computers and physical transmission infrastructure communicating via the standardized **TCP/IP protocol suite**. It relies on packet switching to route discrete datagrams across fiber-optic cables, repeaters, satellite links, and enterprise routers.

TECHNICAL DEFINITION: THE WORLD WIDE WEB (WWW)

An abstract cyberspace information retrieval application that operates on top of the physical Internet infrastructure. It is composed of interlinked hypermedia documents (webpages) interconnected via hyperlinks, addressed globally by **Uniform Resource Locators (URLs)**, and transferred using **HTTP/HTTPS**.

2. Exhaustive Comparison: Internet vs. World Wide Web (12-Point Analysis)

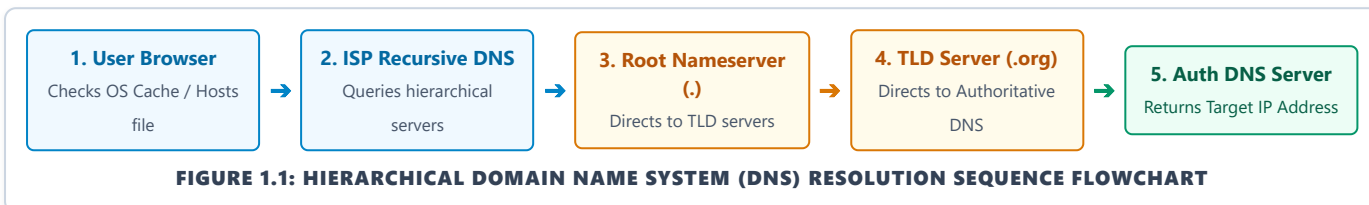
#	Dimension	The Internet	The World Wide Web (WWW)
1	Fundamental Nature	A vast physical and logical network of interconnected hardware devices, cables, and routers.	An abstract cyberspace collection of hyperlinked multimedia documents and online resources.
2	Year of Origin	1969 (Launch of ARPANET by the US Department of Defense).	1989 (Invented by Sir Tim Berners-Lee at CERN, Switzerland).
3	OSI / TCP-IP Layer	Operates primarily across lower layers: Physical, Data Link, Network, and Transport.	Operates strictly at the topmost layer: the Application Layer (Layer 7).
4	Governing Protocols	Driven by IP $\frac{\text{IPv4}}{\text{IPv6}}$, TCP, UDP, BGP, ICMP, ARP, RIP.	Driven by HTTP, HTTPS, WebSockets, WebRTC.
5	Core Purpose	Provides transport pipelines and routing infrastructure for transferring raw binary packets.	Formats, structures, presents, and navigates multimedia information via hypertext.
6	Dependency Relation	Completely independent. It existed for 20 years before the Web and functions without it.	Completely dependent on the Internet. The Web cannot exist without the Internet backbone.
7	Addressing Mechanism	Hardware MAC Addresses (48-bit) and logical IP Addresses (32-bit IPv4 / 128-bit IPv6).	Uniform Resource Locators (URLs) and URIs (e.g., https://mmumullana.org/index.html).
8	Services Encompassed	Encompasses the Web, Email (SMTP/IMAP), File Transfer (FTP), Shells (SSH), VoIP, Gaming.	Encompasses websites, web applications, web services, REST APIs, and online hypermedia.
9	Client Software Required	Operating system networking stack, terminal clients, socket utilities, FTP clients.	Web Browsers (Google Chrome, Mozilla Firefox, Microsoft Edge, Apple Safari).
10	Physical Hardware	Submarine fiber cables, repeaters, cellular towers, ISP routers, satellite transponders.	Web hosting server machines, disk arrays, HTTP software daemons, content databases.
11	Central Concept	Packet switching, packet headers, hop-by-hop routing, error checking, congestion control.	Hypertext, hyperlinks, DOM trees, CSS cascades, client-side rendering engines.
12	Governing Entities	IETF (Internet Engineering Task Force), ICANN, IANA, ITU.	W3C (World Wide Web Consortium), WHATWG.

3. Evolution and History of the World Wide Web

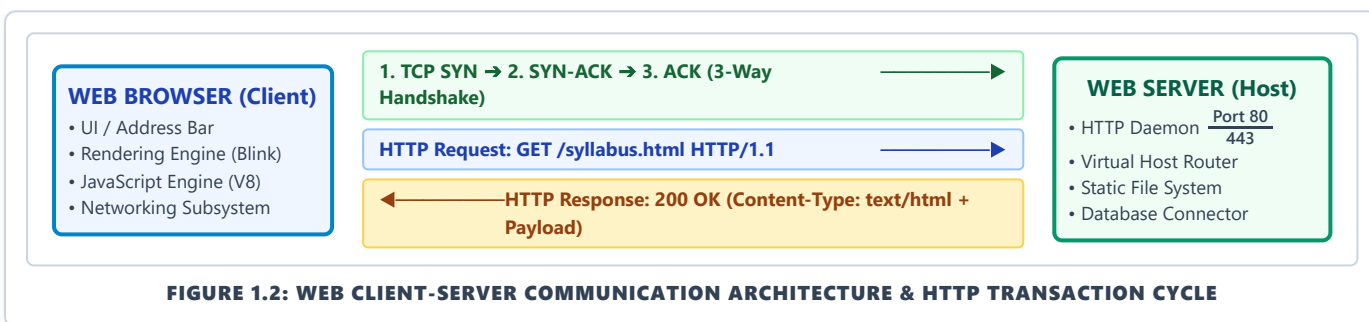
- **1969 (ARPANET):** Funded by the US Defense Advanced Research Projects Agency (DARPA), connecting 4 university nodes (UCLA, Stanford, UC Santa Barbara, Utah) via Interface Message Processors (IMPs).
- **January 1, 1983 ("Flag Day"):** ARPANET officially switched protocols from NCP to **TCP/IP**, establishing the architecture of the modern Internet.

- **1989–1990 (Tim Berners-Lee at CERN):** Sir Tim Berners-Lee proposed an information management system using hypertext. He developed the first web server (CERN httpd), the first web browser (WorldWideWeb), HTML, and HTTP.
- **1993 (NCSA Mosaic):** Developed by Marc Andreessen; popularized the Web by rendering images inline with text.
- **1994 (Formation of W3C):** Tim Berners-Lee founded the World Wide Web Consortium to maintain standardized specifications.
- **Web Eras:**
 - *Web 1.0 (1991–2004, "Read-Only Web"):* Static HTML pages, table-based layouts, centralized content with no interaction.
 - *Web 2.0 (2004–2016, "Read-Write / Social Web"):* User-generated content, social networks, AJAX asynchronous updates, and responsive design.
 - *Web 3.0 (2016–Present, "Semantic & Decentralized Web"):* Machine-readable metadata, AI search, decentralized ledger technologies, and PWAs.

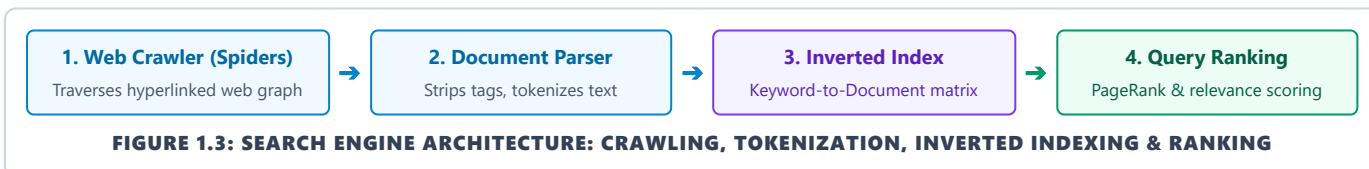
4. Requirements for Working with Internet and WWW: Hierarchical DNS Resolution



5. Web Browsers and Web Servers: Architectural Breakdown



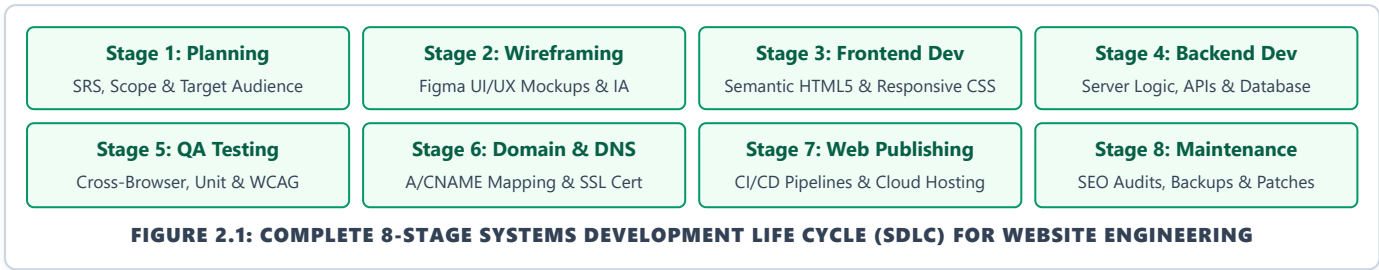
6. Searching and Web-Casting Techniques



UNIT II: BASICS OF HTML & WEBSITE ENGINEERING

Website Lifecycle, Hosting, Static vs Dynamic, HTML Structure, Tags, Lists, Hyperlinks, Images, Image Mapping & IFrames (6 Hours)

1. Steps for Developing a Website (Complete 8-Stage Lifecycle Flowchart)



2. Static vs. Dynamic Websites: Comprehensive 12-Point Comparison

#	Dimension	Static Website	Dynamic Website
1	Core Definition	Pre-authored, fixed content stored on disk; appears identical to every visitor.	Content generated on-the-fly based on client request, user profile, and database queries.
2	Technologies Employed	Pure HTML5, CSS3, and basic vanilla client-side JavaScript.	Server-side runtimes (PHP, Node.js, Python, Java) paired with relational or NoSQL databases.
3	Database Dependency	Zero database requirement. Text and media paths are hardcoded directly into files.	Heavily database-driven (MySQL, PostgreSQL, MongoDB, Oracle).
4	Server Processing Overhead	Negligible CPU overhead. The web server performs direct disk I/O to socket transmission.	High CPU/RAM utilization. Executes scripts, authenticates sessions, runs SQL queries.
5	Page Load Performance	Extremely fast due to lightweight payload and edge CDN caching worldwide.	Slightly higher latency due to server-side script execution and database round-trips.
6	Development Complexity	Low architectural complexity; easy for beginners to author and deploy.	High complexity; requires full-stack architectures, ORMs, and secure state handling.
7	Hosting & Maintenance Cost	Near zero. Can be hosted free on static platforms (GitHub Pages, Netlify, Cloudflare).	Higher recurring costs for virtual servers, database storage, and auto-scaling compute.
8	Ease of Content Updating	Difficult at scale. Changing a navigation menu requires updating every single HTML file.	Effortless for non-technical users via CMS portals (WordPress, Strapi) and admin panels.
9	User Interactivity & Auth	Limited to client interactions; cannot provide user accounts or persistent shopping carts.	Full interactive capability: secure user login, payment checkout, customized feeds.
10	Security Vulnerability Surface	Immune to server-side exploits like SQL Injection and Remote Code Execution (RCE).	Exposed to SQL Injection, Cross-Site Scripting (XSS), CSRF, and broken access controls.
11	File Extensions	Files saved strictly as .html or .htm.	Historically saved as .php, .jsp, .aspx, or routed dynamic endpoints.
12	Real-World Examples	Documentation sites, personal portfolios, restaurant menus, landing pages.	Amazon, Netflix, Twitter/X, Banking portals, MMU Student ERP System.

3. HTML Core Tags & Client-Side Image Mapping

A. Headings, Paragraphs, Line Breaks & Rules

```
<h1>Main Title (Level 1)</h1> <h2>Sub-Unit Header (Level 2)</h2> <p>Paragraph tag defines standard blocks of text.<br>The br tag forces an immediate line break.</p> <hr> <!-- Horizontal thematic break rule -->
```

B. Text Formatting & Semantic Styles

- `<b>` vs. `<strong>`: `<b>` applies physical bold styling; `<strong>` conveys semantic importance for screen readers.
- `<i>` vs. `<em>`: `<i>` applies physical italics; `<em>` marks emphasized phonetic stress.
- `<mark>` (yellow highlight), `<del>` (strikethrough), `<ins>` (underlined insert), `<sup>` (x^2), `<sub>` (H_2O).

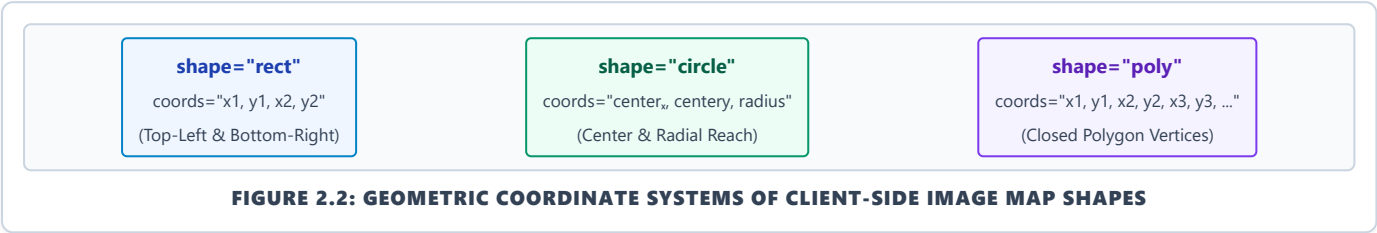
C. Ordered, Unordered, and Definition Lists

```
<ul type="square"> <li>Computer Networks</li> <li>Operating Systems</li> </ul> <ol type="I" start="3"> <li>Unit III: HTML5</li> <li>Unit IV: CSS</li> </ol> <dl> <dt>HTML</dt><dd>Hypertext Markup Language for structuring web documents.</dd> <dt>CSS</dt><dd>Cascading Style Sheets for describing visual presentation.</dd> </dl>
```

D. Hyperlinks (<a> Anchor Tag)

```
<!-- 1. External link opening securely in a new browser tab --> <a href="https://www.w3.org" target="_blank" rel="noopener noreferrer">Visit W3C Official</a> <!-- 2. Internal bookmark anchor jump --> <a href="#unit4-css">Jump directly to Unit IV Notes</a> <!-- 3. Email and Telephone protocols --> <a href="mailto:contact@mmumullana.org">Email University</a> <a href="tel:+911731274475">Call Office</a>
```

E. Client-Side Image Mapping (<map> and <area>)



```
 <map name="boardmap"> <area shape="rect" coords="20,20,180,180" href="cpu.html" alt="CPU Socket Details"> <area shape="circle" coords="250,150,45" href="cmos.html" alt="CMOS Battery Info"> <area shape="poly" coords="300,50,380,50,380,120,300,120" href="ram.html" alt="RAM Slots"> </map>
```

UNIT III: HTML5 & MODERN WEB APPLICATION APIS

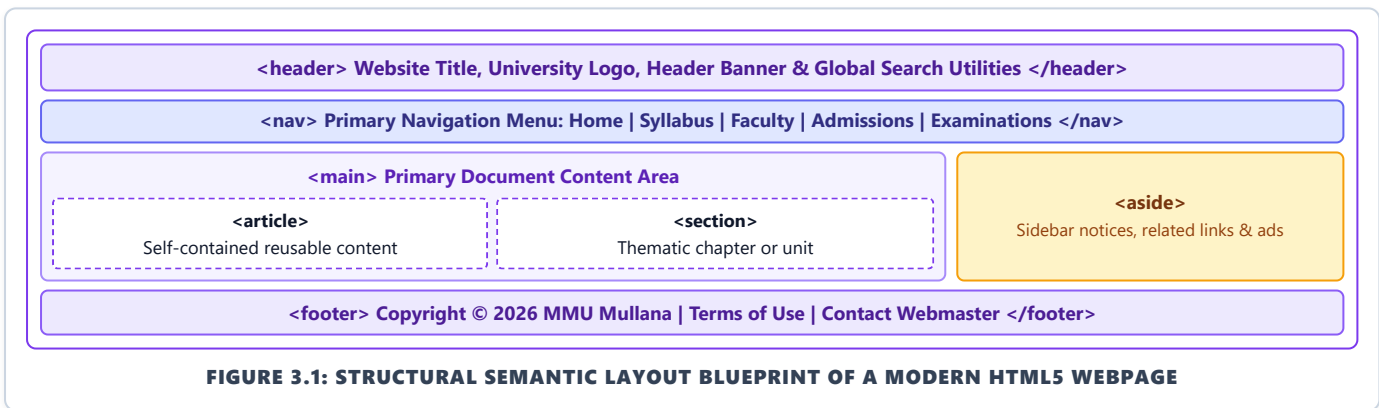
HTML5 Advantages vs HTML4, Semantic Layout Blueprint, Native Audio/Video, Canvas 2D, Web Storage & Modern Forms (6 Hours)

1. Introduction, Evolution & Advantages of HTML5 over HTML4

Ratified as an official W3C Recommendation in October 2014, **HTML5** modernized web engineering from simple document formatting into a powerful application runtime.

HTML4 Limitation	Problem in HTML4	HTML5 Modern Solution
Lack of Semantics	Heavy reliance on meaningless <code><div class="header"></code> and "div-soup".	Native structural semantic elements (<code><header></code> , <code><nav></code> , <code><main></code> , <code><article></code>).
Multimedia Plugins	Required insecure, crash-prone third-party binary plugins (Adobe Flash, Silverlight).	Native <code><video></code> and <code><audio></code> hardware-accelerated playback with multi-codec fallbacks.
Client-Side Storage	Limited solely to 4KB HTTP Cookies transmitted with every HTTP request.	Web Storage API (<code>localStorage</code> and <code>sessionStorage</code>) offering 5MB–10MB isolated storage.
Procedural Graphics	Zero native 2D drawing; required pre-baked images or Flash applets.	High-performance scriptable <code><canvas></code> 2D context API and SVG integration.
Form Validation	Required extensive custom JavaScript regular expressions for email, dates, and numbers.	Native form controls (<code>type="email"</code> , <code>type="date"</code> , <code>required</code> , <code>pattern</code>).

2. HTML5 Structural Semantic Elements Layout Blueprint



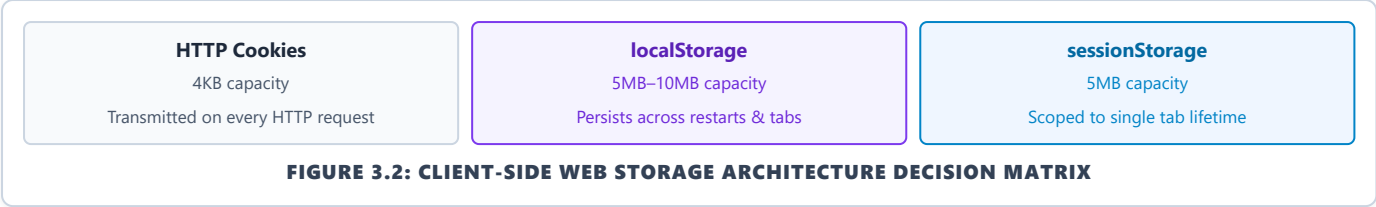
3. HTML5 Native Multimedia (<video> and <audio> APIs)

```
<!-- Native HTML5 Video with Multi-Codec Fallback & Subtitles --> <video width="640" height="360" controls
poster="poster_image.jpg" preload="metadata"> <source src="lecture.mp4" type="video/mp4"> <source
src="lecture.webm" type="video/webm"> <track src="subtitles_en.vtt" kind="subtitles" srclang="en"
label="English"> Your browser does not support native HTML5 video playback. </video> <!-- Native HTML5 Audio
Element --> <audio controls preload="auto"> <source src="podcast.mp3" type="audio/mpeg"> <source
src="podcast.ogg" type="audio/ogg"> Your browser does not support the audio element. </audio>
```

4. HTML5 Canvas API & 2D Context Drawing

```
<canvas id="myCanvas" width="400" height="160" style="border:1px solid #cbd5e1;"></canvas> <script> const canvas
= document.getElementById("myCanvas"); const ctx = canvas.getContext("2d"); // 1. Draw Rectangular Bar
ctx.fillStyle = "#0284c7"; ctx.fillRect(40, 40, 60, 100); // 2. Draw Radial Arc / Circle ctx.beginPath();
ctx.arc(220, 90, 40, 0, 2 * Math.PI); ctx.fillStyle = "#f59e0b"; ctx.fill(); ctx.stroke(); // 3. Render Text
Label ctx.font = "bold 13px sans-serif"; ctx.fillStyle = "#0f172a"; ctx.fillText("Canvas 2D API", 40, 25);
</script>
```

5. HTML5 Web Storage: Cookies vs. LocalStorage vs. SessionStorage



UNIT IV: CASCADEING STYLE SHEETS (CSS) ARCHITECTURE

Cascade & Specificity, 3 Applying Methods, Box Model, Display, 5 Positioning Schemes, Floats & Tables (6 Hours)

1. Introduction to Style Sheets, Cascade & Specificity

CSS SPECIFICITY FORMULA (WEIGHT MATRIX)

Specificity determines which CSS rule takes precedence when multiple selectors target the same HTML element:

$$\text{ext(Specificity)} = (\text{ext(Inline Style) [1000]}) + (\text{ext(IDs) [0100]}) + (\text{ext(Classes/Pseudo) [0010]}) + (\text{ext(Elements/Tags) [0001]}) \cdot$$

!important overrides standard cascade weights and must be used with caution.

- *Inheritance*: Typography properties (color, font-family) inherit to children; Box Model properties (margin, padding, border) do not inherit.

2. Three Methods of Applying CSS (Comparison Table)

Method	Syntax Example	Scope	Evaluation & Browser Caching
Inline CSS	<code><p style="color: blue;"></code>	Applies strictly to that single element.	Worst practice. Pollutes HTML markup; cannot be cached separately by browsers.
Internal / Embedded	<code><style> body { margin:0; } </style></code>	Applies to the entire single HTML document.	Acceptable for single-page documents; cannot be reused across other pages.
External CSS	<code><link rel="stylesheet" href="style.css"></code>	Styles entire multi-page application uniformly.	Industry Standard : Full separation of concerns; cached by browser across all pages.

3. Text Formatting, Typography & Color Gradients

```
.academic-typography { font-family: 'Segoe UI', Arial, sans-serif; font-size: 16px; font-weight: 600; line-height: 1.5; /* Proportional line height */ letter-spacing: 0.5px; /* Character spacing */ text-transform: capitalize; /* uppercase | lowercase | capitalize */ text-align: justify; /* left | right | center | justify */ text-shadow: 1px 1px 2px rgba(0, 0, 0, 0.2); } /* Linear & Radial Gradient Backgrounds */ .gradient-banner { background: linear-gradient(135deg, #0284c7 0%, #1e1b4b 100%); color: #ffffff; padding: 16px; border-radius: 6px; } .radial-spotlight { background: radial-gradient(circle at center, #f59e0b 0%, #9a3412 100%); color: #ffffff; padding: 16px; border-radius: 6px; }
```

4. The Standard CSS Box Model & Sizing Reset

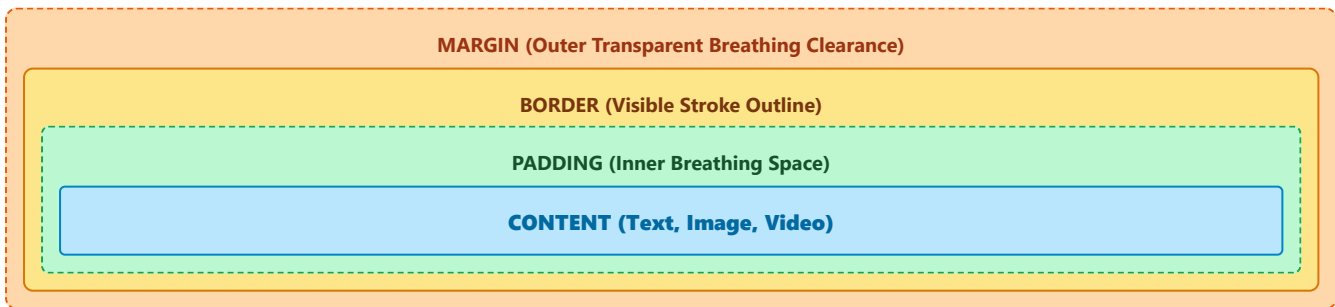
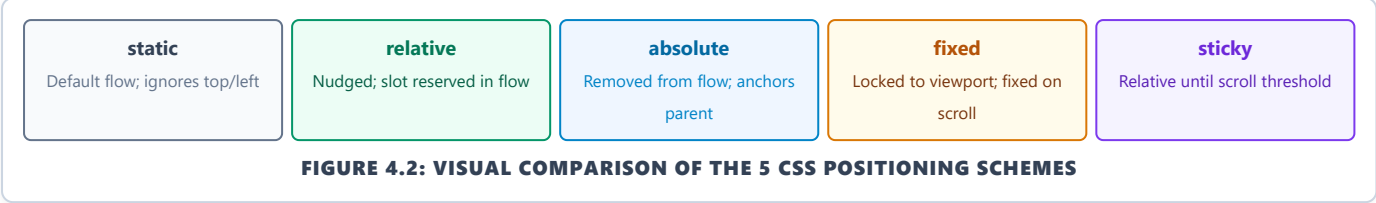


FIGURE 4.1: STRUCTURAL ARCHITECTURE OF THE STANDARD CSS BOX MODEL

```
/* Universal Box-Sizing Reset */ *, *::before, *::after { box-sizing: border-box; /* Total Width = Specified Width (No blowouts!) */ }
```

5. CSS 5 Positioning Schemes Comparison



6. Floats, Micro-Clearfix & Zebra Tables

```
/* Micro-Clearfix Hack preventing parent container collapse */ .clearfix::after { content: ""; display: table; clear: both; } /* Professional Zebra-Striped Data Table */ table.university-table { width: 100%; border-collapse: collapse; /* Merges double borders */ } table.university-table th, table.university-table td { border: 1px solid #cbd5e1; padding: 8px 12px; } table.university-table th { background-color: #1e293b; color: #ffffff; } table.university-table tr:nth-child(even) { background-color: #f8fafc; /* Alternating row */ }
```

UNIVERSITY EXAMINATION SOLVED QUESTIONS WITH SCHEMATICS

Structured strictly according to MMDU Examination Pattern: Section A (1M), Section B (2M), Section C (4M), and Section D (6M) with Diagrams in BOTH Questions and Solutions

The following solved questions strictly enforce the user's mandatory design rule: **every question features an architectural schematic in the problem statement AND an analytical diagram in the step-by-step solution.**

SECTION A: SHORT CONCEPTUAL QUESTIONS (1 MARK EACH)

Q1. [Unit I - DNS Hierarchy]: State the primary purpose of the Root Name Server in the DNS resolution hierarchy shown in Figure Q1. [1 Mark]

Section
A

Root (.) → TLD (.org/.com) → Auth DNS

FIGURE Q1: DNS HIERARCHICAL TREE QUESTION DIAGRAM

Answer Vector: Root (.) redirects resolver to specific TLD nameservers.

FIGURE S1: DNS ROOT REDIRECTION VECTOR

STEP-BY-STEP SOLUTION:

The **Root Name Server (.)** stands at the apex of the DNS hierarchy. It does not store individual host IPs; rather, it reads the Top-Level Domain of the queried domain (e.g., .org in mmumullana.org) and returns the IP addresses of the designated **TLD Nameservers** to the recursive resolver.

Q2. [Unit II - HTML Image Map]: Given the rectangular clickable area shown in Figure Q2, write the exact HTML <area> tag syntax. [1 Mark]

Section
A

Rect Top-Left: (20, 30) | Bottom-Right: (150, 120) → Target: info.html

FIGURE Q2: COORDINATE PLANE QUESTION DIAGRAM

`<area shape="rect" coords="20,30,150,120" href="info.html">`

FIGURE S2: AREA TAG MAPPING SOLUTION DIAGRAM

STEP-BY-STEP SOLUTION:

For a rectangular shape (shape="rect"), the coords attribute accepts four comma-separated integers representing (x₁, y₁, x₂, y₂):
`<area shape="rect" coords="20,30,150,120" href="info.html" alt="Information Portal">`

Q3. [Unit III - Web Storage]: Differentiate the lifetime of localStorage vs. sessionStorage shown in Figure Q3. [1 Mark]

Section
A

Storage Node → [Branch A: Tab Close] vs. [Branch B: Browser Restart]

FIGURE Q3: PERSISTENCE LIFECYCLE QUESTION DIAGRAM

sessionStorage: Tab Lifetime | localStorage: Permanent

FIGURE S3: PERSISTENCE RESOLUTION SOLUTION DIAGRAM

STEP-BY-STEP SOLUTION:

Data stored in **localStorage** persists indefinitely until explicitly cleared via script or user action (even across browser restarts). In contrast, **sessionStorage** is strictly scoped to the active browser tab session and is instantly purged when the tab is closed.

Q4. [Unit IV - CSS Specificity]: Calculate the total specificity score of the selector in Figure Q4. [1 Mark] Section A

Selector: `nav#primary-menu ul.dropdown > li a:hover`

FIGURE Q4: COMPOUND SELECTOR BREAKDOWN DIAGRAM

$\text{ext}(\text{IDs})=1, \text{ext}(\text{Classes/Pseudo})=2, \text{ext}(\text{Tags})=3 \Rightarrow 0123$

FIGURE S4: SPECIFICITY SUMMATION MATRIX

STEP-BY-STEP SOLUTION:

Counting selector components:

- Inline styles = 0
- ID selectors (`#primary-menu`) = 1 times 100 = 100
- Classes & Pseudo-classes (`.dropdown`, `:hover`) = 2 times 10 = 20
- Elements / Tags (`nav`, `ul`, `li`, `a`) = 4 times 1 = 4

Total Specificity Score = 0124 (or (0, 1, 2, 4)).

SECTION B: ANALYTICAL CONCEPT QUESTIONS (2 MARKS EACH)

Q5. [Unit I - TCP/IP Protocol Stack]: Illustrate the 3-way handshake sequence when a browser initiates an HTTP connection to a web server. [2 Marks]

Section B

Client [State: CLOSED] → Server [State: LISTEN, Port 80]

FIGURE Q5: CONNECTION ENDPOINTS SCHEMATIC

1. ext(SYN)(x) → | ← 2. ext(SYN)(y)+ ext(ACK)(x+1) | 3. ext(ACK)(y+1) →

FIGURE S5: TCP 3-WAY HANDSHAKE TIMING SEQUENCE

STEP-BY-STEP SOLUTION:

- 1. SYN (Synchronize):** The client browser generates a randomized initial sequence number (ISN) x and transmits a TCP packet with the SYN control flag set to the server.
- 2. SYN-ACK (Acknowledge):** The web server receives the SYN, allocates socket buffers, generates its own ISN y , and transmits back a packet with SYN and $ACK = x + 1$.
- 3. ACK (Established):** The client acknowledges receipt with $ACK = y + 1$. Both sides enter the ESTABLISHED state and HTTP data transfer begins.

Q6. [Unit II - HTML5 IFrames vs Legacy Frames]: Why are HTML4 <frameset> elements deprecated, and how does <iframe> solve these issues? [2 Marks]

Section B

Legacy: <frameset cols="20%,80%"> (Replaces <body>, breaks URLs)

FIGURE Q6: RIGID FRAMESET ARCHITECTURE DIAGRAM

Modern: <iframe sandbox="..."> (Embedded anywhere in normal DOM)

FIGURE S6: INLINE FRAME SANDBOXED ARCHITECTURE

STEP-BY-STEP SOLUTION:

- **Why <frameset> was Deprecated:** <frameset> replaced the document <body>, completely breaking browser bookmarking, back/forward navigation history, print styles, and search engine crawling.
- **The <iframe> Modern Solution:** The Inline Frame (<iframe>) lives harmoniously inside the normal HTML5 <body>. It supports security isolation via the sandbox attribute (e.g., `sandbox="allow-scripts"`) and supports responsive CSS positioning.

Q7. [Unit III - Canvas 2D Context]: Write the JavaScript instructions to draw a green rectangle of width 80px, height 40px at (30, 20) on a <canvas>. [2 Marks]

Section B

Canvas Coordinate System: Origin (0,0) at Top-Left → Point (30,20)

FIGURE Q7: CANVAS COORDINATE GRID DIAGRAM

Rendered Green Box: $x=30, y=20, w=80, h=40$

FIGURE S7: RASTER EXECUTION OUTPUT DIAGRAM

STEP-BY-STEP SOLUTION:

```
const canvas = document.getElementById("myCanvas"); const ctx = canvas.getContext("2d"); ctx.fillStyle = "#10b981"; // Emerald green fill ctx.fillRect(30, 20, 80, 40); // fillRect(x, y, width, height)
```

Q8. [Unit IV - CSS Box Sizing]: Compare content-box vs border-box calculations for an element with width 200px, padding 20px, and border 5px. [2 Marks] **Section B**

Specified: $\text{ext}(\text{Width}) = 200 \text{ ext}(\text{px})$, $\text{ext}(\text{Padding}) = 20 \text{ ext}(\text{px})$, $\text{ext}(\text{Border}) = 5 \text{ ext}(\text{px})$

FIGURE Q8: DIMENSIONAL PARAMETERS DIAGRAM

$\text{ext}(\text{content-box}) = 250 \text{ ext}(\text{px})$ vs. $\text{ext}(\text{border-box}) = 200 \text{ ext}(\text{px})$

FIGURE S8: BOX-SIZING FORMULA COMPARISON DIAGRAM

STEP-BY-STEP SOLUTION:

1. Standard content-box:

$\text{ext}(\text{Total Rendered Width}) = \text{ext}(\text{Width}) + 2 \text{ times } (\text{ext}(\text{Padding})) + 2 \text{ times } (\text{ext}(\text{Border})) = 200 + 40 + 10 = 250 \text{ ext}(\text{px})$

2. Modern border-box:

$\text{ext}(\text{Total Rendered Width}) = \text{ext}(\text{Specified Width}) = 200 \text{ ext}(\text{px})$

The browser automatically shrinks the inner content area to $200 - 40 - 10 = 150 \text{ ext}(\text{px})$, preventing layout blowouts.

SECTION C: DESCRIPTIVE APPLICATION PROBLEMS (4 MARKS EACH)

Q9. [Unit I - Search Engine Architecture]: Explain the 4 primary subsystems of a web search engine and detail how inverted indexing works. [4 Marks]

Section C

Raw Hyperlinked Web → [Crawler Spiders] → [Document Store]

FIGURE Q9: WEB INGESTION ARCHITECTURE DIAGRAM

Word: "HTML" → Doc IDs: [Doc 101, Doc 104, Doc 209]

FIGURE S9: INVERTED INDEX POSTINGS LIST DIAGRAM

STEP-BY-STEP SOLUTION:

A modern search engine functions via four tightly integrated subsystems:

- Web Crawler (Spider/Bot):** Recursively fetches web documents across the global Internet by discovering and following hyperlinks starting from seed URLs.
- Document Parser & Tokenizer:** Strips HTML tags, converts characters to lowercase, eliminates non-informative stop-words (e.g., "the", "and"), and stems words to their linguistic roots.
- Inverted Indexer:** Rather than indexing documents and listing words inside them, it creates a dictionary of all distinct words mapped to their containing Document IDs and positional frequencies (Postings List).
- Query Processor & Ranking Engine (PageRank / AI):** Evaluates user search keywords against the Inverted Index, computes relevance via TF-IDF and backlink authority graphs (PageRank), and delivers ranked results within milliseconds.

Q10. [Unit II - HTML Tables & Forms]: Write standards-compliant HTML markup to render the university course fee schedule shown in Figure Q10. [4 Marks]

Section C

Columns: [Course Code | Branch Name | Semester Fee] with Colspan/Rowspan

FIGURE Q10: TARGET TABLE WIREFRAME DIAGRAM

HTML Table Structure: <table><thead><tbody><tfoot>

FIGURE S10: SEMANTIC TABLE HIERARCHY DIAGRAM

STEP-BY-STEP SOLUTION:

```
<table border="1" cellpadding="8" cellspacing="0" style="border-collapse:collapse; width:100%;> <thead
style="background:#1e293b; color:#fff;"> <tr> <th>Course Code</th> <th>Engineering Discipline</th>
<th>Semester Fee (INR)</th> </tr> </thead> <tbody> <tr> <td rowspan="2" align="center"><strong>BCSE-
012</strong></td> <td>Computer Science & Engineering</td> <td align="right">₹75,000</td> </tr> <tr>
<td>Information Technology</td> <td align="right">₹75,000</td> </tr> </tbody> <tfoot
style="background:#f1f5f9; font-weight:bold;"> <tr> <td colspan="2" align="right">Total Tuition Fee:</td>
<td align="right">₹1,50,000</td> </tr> </tfoot> </table>
```

Q11. [Unit III - HTML5 Semantic Layout]: Construct a complete semantic page wireframe utilizing <header>, <nav>, <main>, <article>, <aside>, and <footer>. [4 Marks]

Section C

Wireframe: Top Banner → Navbar → [Left: Articles | Right: Aside] → Footer

FIGURE Q11: SEMANTIC PAGE WIREFRAME PROBLEM DIAGRAM

DOM Blueprint: Root → Header → Nav → Main (Article+Section) + Aside
→ Footer

FIGURE S11: DOM TREE SEMANTIC ARCHITECTURE DIAGRAM

STEP-BY-STEP SOLUTION:

```
<!DOCTYPE html> <html lang="en"> <head> <meta charset="UTF-8"><title>MMU Engineering Portal</title> </head>
<body> <header> <h1>Maharishi Markandeshwar University</h1> </header> <nav> <a href="#home">Home</a> | <a
href="#academics">Academics</a> | <a href="#contact">Contact</a> </nav> <main> <article> <h2>B.Tech 1st
Year Syllabus Update</h2> <p>BCSE-012 Fundamental of Web Technologies notes are now available.</p>
</article> </main> <aside> <h3>Examination Notices</h3> <p>Mid-term practical tests commence next week.</p>
</aside> <footer> <p>&copy; 2026 MMU Mullana. All rights reserved.</p> </footer> </body> </html>
```

Reference Point A: Nearest Non-Static Ancestor vs. Reference Point B: Viewport

FIGURE Q12: REFERENCE COORDINATE ORIGIN DIAGRAM

Absolute: Scrolls with document | Fixed: Remains locked on screen

FIGURE S12: POSITIONING SCROLL PHYSICS DIAGRAM

STEP-BY-STEP SOLUTION:

Criterion	position: absolute	position: fixed
Coordinate Origin Anchor	Offsets (top, left) calculated relative to nearest positioned (non-static) ancestor.	Offsets calculated strictly relative to browser Viewport window.
Scroll Behavior	Scrolls naturally along with document body when user scrolls.	Remains completely fixed/frozen on screen during scrolling.
Document Flow Impact	Completely pulled out of normal flow; zero space reserved.	Completely pulled out of normal flow; zero space reserved.
Typical Real-World Use Case	Dropdown menu badges, tooltips, custom card overlays.	Persistent navigation top-bars, floating WhatsApp/chat widgets.

SECTION D: COMPREHENSIVE ARCHITECTURAL DESIGN PROBLEMS (6 MARKS EACH)

Section
D

Q13. [Unit I - Master Architecture]: Trace the complete technical lifecycle when an engineering student types "https://mmumullana.org/index.html" into a web browser, from DNS resolution to HTTP rendering. [6 Marks]

Input URL: https://mmumullana.org/index.html → Browser Address Bar

FIGURE Q13: USER NAVIGATION TRIGGER DIAGRAM

Phase 1: DNS → Phase 2: TCP/TLS → Phase 3: HTTP → Phase 4: DOM/CSSOM

FIGURE S13: END-TO-END TRANSACTION PIPELINE DIAGRAM

STEP-BY-STEP SOLUTION:

The complete transaction proceeds across 4 architectural phases:

Phase 1: URL Parsing & DNS Resolution

- 1. Browser parses the protocol (https), hostname (mmumullana.org), and resource path (/index.html).
- 2. Browser checks its internal DNS cache, followed by the OS cache (and hosts file). If missing, it dispatches a UDP query on Port 53 to the ISP Recursive Resolver.
- 3. The resolver queries the Root Nameserver (.), which directs it to the .org TLD nameserver.
- 4. The TLD server directs it to MMU's Authoritative Nameserver, which returns the server's public IP address (e.g., 104.21.48.12).

Phase 2: TCP Handshake & TLS Encryption

- 1. The browser initiates a TCP 3-way handshake (SYN → SYN-ACK → ACK) on Port 443.
- 2. A TLS cryptographic handshake occurs: cipher suites are negotiated, the server presents its X.509 SSL certificate, session keys are derived via asymmetric cryptography, and symmetric encryption begins.

Phase 3: HTTP Request-Response Transaction

- 1. Browser transmits: GET /index.html HTTP/1.1 Host: mmumullana.org Accept: text/html .
- 2. The web server (NGINX/Apache) reads the file from storage and returns: HTTP/1.1 200 OK Content-Type: text/html Content-Length: 4210 [HTML Body Payload].

Phase 4: Browser Engine Rendering Pipeline

- 1. **DOM Construction:** HTML tokens are converted into DOM node hierarchy.
- 2. **CSSOM Construction:** Linked stylesheets are parsed into the CSS Object Model.
- 3. **Render Tree:** DOM + CSSOM combine into visible nodes.
- 4. **Layout & Paint:** Exact pixel geometry is computed and painted on screen via the graphics subsystem.

Q14. [Unit II & IV - Full-Page Website Layout]: Design a complete, responsive university homepage layout with a horizontal navigation bar, a floating 2-column content area, and a persistent footer using CSS. [6 Marks]

Section D

Layout Blueprint: Navbar (Flex) + [Sidebar: 30% | Main: 68%] + Micro-Clearfix + Footer

FIGURE Q14: MULTI-COLUMN WIREFRAME SCHEMATIC DIAGRAM

Layout Grid Solution: Float-based architecture with Clearfix container

FIGURE S14: FLOAT & CLEARFIX PHYSICS ARCHITECTURE DIAGRAM

STEP-BY-STEP SOLUTION:

```
<!DOCTYPE html> <html lang="en"> <head> <meta charset="UTF-8"><meta name="viewport" content="width=device-width, initial-scale=1.0"> <title>MMU Mullana - Department of CSE</title> <style> *, *::before, *::after { box-sizing: border-box; margin:0; padding:0; } body { font-family: Segoe UI, sans-serif; background: #f8fafc; color: #0f172a; } /* Horizontal Navbar */ nav.top-nav { background: #0f172a; } nav.top-nav ul { list-style: none; display: flex; } nav.top-nav ul li a { display: block; color: #ffffff; padding: 12px 20px; text-decoration: none; } nav.top-nav ul li a:hover { background: #0284c7; } /* Micro-Clearfix Layout Container */ .container { width: 90%; max-width: 1100px; margin: 20px auto; } .clearfix::after { content: ""; display: table; clear: both; } /* Floating Columns */ .sidebar { float: left; width: 28%; background: #ffffff; padding: 16px; border: 1px solid #cbd5e1; border-radius: 4px; } .main-content { float: right; width: 69%; background: #ffffff; padding: 16px; border: 1px solid #cbd5e1; border-radius: 4px; } /* Footer */ footer.site-footer { background: #1e293b; color: #94a3b8; text-align: center; padding: 14px; margin-top: 20px; } </style> </head> <body> <nav class="top-nav"> <ul> <li><a href="#">Home</a></li> <li><a href="#">Admissions</a></li> <li><a href="#">Syllabus</a></li> <li><a href="#">Placements</a></li> </ul> </nav> <div class="container clearfix"> <aside class="sidebar"> <h3>Quick Links</h3> <p>Download BCSE-012 Examination Sheet.</p> </aside> <main class="main-content"> <h2>Department of Computer Science & Engineering</h2> <p>Accredited Grade A++ by NAAC. Offering premier technical education in Web Technologies, AI, and Cybersecurity.</p> </main> </div> <footer class="site-footer"> <p>&copy; 2026 Maharishi Markandeshwar (Deemed to be University), Mullana.</p> </footer> </body> </html>
```

Q15. [Unit III - Dynamic Canvas Rendering]: Develop complete HTML5 & JavaScript code to render an academic marks comparison bar chart with custom labels and colors on a <canvas>. [6 Marks]

Section D

Chart Target: 3 Bars [Math: 85 | BEEE: 90 | WebTech: 95] with Baseline Axis

FIGURE Q15: CANVAS RENDERING PROBLEM SPECIFICATION

Canvas Transform: Baseline at y=160, Scale = 1.4 ext(px)/ ext(mark)

FIGURE S15: COORDINATE TRANSLATION MATRIX DIAGRAM

STEP-BY-STEP SOLUTION:

```
<canvas id="chart" width="450" height="200" style="background:#f8fafc; border:1px solid #cbd5e1;"></canvas>
<script> const canvas = document.getElementById("chart"); const ctx = canvas.getContext("2d"); const
subjects = [ { name: "Math-I", marks: 85, color: "#0284c7" }, { name: "BEEE", marks: 90, color: "#10b981"
}, { name: "WebTech", marks: 95, color: "#7c3aed" } ]; const startX = 60; const barWidth = 60; const gap =
50; const baselineY = 160; // Draw Baseline Axis ctx.beginPath(); ctx.moveTo(30, baselineY);
ctx.lineTo(420, baselineY); ctx.strokeStyle = "#475569"; ctx.lineWidth = 2; ctx.stroke(); // Render Dynamic
Bars & Labels subjects.forEach((sub, i) => { const x = startX + i * (barWidth + gap); const barHeight =
sub.marks * 1.3; const y = baselineY - barHeight; // Fill Bar ctx.fillStyle = sub.color; ctx.fillRect(x, y,
barWidth, barHeight); // Draw Numeric Score on top ctx.fillStyle = "#0f172a"; ctx.font = "bold 12px sans-
serif"; ctx.textAlign = "center"; ctx.fillText(sub.marks + "%", x + barWidth / 2, y - 6); // Draw Subject
Label below axis ctx.font = "600 11px sans-serif"; ctx.fillStyle = "#334155"; ctx.fillText(sub.name, x +
barWidth / 2, baselineY + 18); }); </script>
```